# UNIVERSITI TEKNOLOGI MALAYSIA

## Faculty of Computing

**SEMESTER 1 SESI 2024 / 2025**

# LAB ACTIVITY 2

**SECP 2523 DATABASE (WBL)**


**AHLI KUMPULAN :**

1. EVELYN GOH YUAN QI A23CS0222
2. JOANNE CHING YIN XUAN A23CS0227
3. LIM YU HAN A23CS0241

Let’s assume you have a simple database for an **online bookstore** with the following tables:

- **books**: Stores book information.
    - `book_id (INT)`
    - `title (VARCHAR)`
    - `author (VARCHAR)`
    - `genre (VARCHAR)`
    - `price (DECIMAL)`
    - `published_date (DATE)`
- **customers**: Stores customer information.
    - `customer_id (INT)`
    - `first_name (VARCHAR)`
    - `last_name (VARCHAR)`
    - `email (VARCHAR)`
    - `registration_date (DATE)`
- **orders**: Stores order information.
    - `order_id (INT)`
    - `customer_id (INT) (foreign key from customers)`
    - `order_date (DATE)`
- **order_items**: Stores details of books ordered.
    - `order_item_id (INT)`
    - `order_id (INT) (foreign key from orders)`
    - `book_id (INT) (foreign key from books)`
    - `quantity (INT)`

---

**Questions:**

**Question 1: Retrieve All Books with Their Prices and Published Dates**

Write an SQL query to retrieve the title, price, and published_date of all books in the database.

```
SELECT title, price, published_date
FROM books;
```

**Question 2: Retrieve Customers Who Registered in the Last 30 Days**

Write an SQL query to fetch the first_name, last_name, and email of all customers who registered in the last 30 days.

```
SELECT first_name, last_name, email
FROM customers
WHERE registration_date >= CURDATE() – INTERVAL 30 DAYS;
```

**Question 3: Find Books by a Specific Author**

Write a query to list the title and price of all books written by the author "J.K. Rowling".

```
SELECT title, price
FROM books
WHERE author = 'J.K. Rowling';
```

**Question 4: List the Total Number of Orders for Each Customer**

Write an SQL query to find the total number of orders placed by each customer. Display their first_name, last_name, and the total_orders.

```
SELECT c.first_name, c.last_name, COUNT(o.order_id) AS total_orders
FROM customers c
LEFT OUTER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

**Question 5: Find the Most Expensive Book**

Write a query to find the title of the most expensive book in the database.

```
SELECT title
FROM books
WHERE price = (
        SELECT MAX(price)
        FROM books
      );
```

**Question 6: Calculate the Total Revenue from All Orders**

Write a query to calculate the total revenue generated by the store, considering the quantity of books ordered and their prices.

```
SELECT SUM(oi.quantity * b.price) AS total_revenue
FROM order_items oi
JOIN books b ON oi.book_id = b.book_id;
```

**Question 7: List the Top 5 Customers Who Have Placed the Most Orders**

Write a query to retrieve the names of the top 5 customers who have placed the most orders, along with their total number of orders.

```
SELECT c.first_name, c.last_name, COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_orders DESC
LIMIT 5;
```

**Question 8: Retrieve Books and Their Total Number of Orders**

Write an SQL query to list the titles of books and the total number of times they have been ordered.

```
SELECT b.title, SUM(oi.quantity) AS total_orders
FROM books b
JOIN order_items oi ON b.book_id = oi.book_id
```

```
GROUP BY b.book_id, b.title;
```

**Question 9: Retrieve Customers Who Have Not Placed Any Orders**

Write a query to find all customers who have never placed an order. Display their first_name, last_name, and email.

```
SELECT first_name, last_name, email
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

**Question 10: Find the Most Popular Genre Based on Orders**

Write an SQL query to find the genre of books that has been ordered the most. Display the genre and the total number of books ordered from that genre.

```
SELECT b.genre, SUM(oi.quantity) AS total_orders
FROM books b
JOIN order_items oi ON b.book_id = oi.book_id
GROUP BY b.genre
ORDER BY total_orders DECS
LIMIT 1;
```

**Question 11: Find Customers Who Have Spent More than $500**

Write a query to find customers who have spent more than $500 on their orders. Display the first_name, last_name, and the total amount spent by each customer.

```
SELECT c.first_name, c.last_name, SUM(b.price * oi.quantity) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN books b ON oi.book_id = b.book_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING total_spent > 500;
```

**Question 12: Find Orders Containing a Specific Book**

Write a query to find all orders that contain the book titled "The Catcher in the Rye". Display the order_id and the order_date.

```
SELECT o.order_id, o.order_date
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN books b ON oi.book_id = b.book_id
WHERE b.title = ‘The Catcher in the Rye’;
```

**Question 13: Find the Average Price of Books by Genre**

Write an SQL query to calculate the average price of books for each genre. Display the genre and the average price.

```
SELECT genre, AVG(price) AS average_price
FROM books
GROUP BY genre;
```

**Question 14: Retrieve Orders Placed in the Last Week**

Write a query to retrieve the order_id and order_date of all orders placed in the last week.

```
SELECT order_id, order_date
FROM orders
WHERE order_date >= CURDATE() – INTERVAL 7 DAY;
```

**Question 15: Retrieve Customers Who Ordered Books by a Specific Author**

Write an SQL query to find the first_name, last_name, and email of customers who have ordered books written by the author "George Orwell".

```
SELECT c.first_name, c.last_name, c.email
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN books b ON oi.book_id = b.book_id
WHERE b.author = ‘George Orwell’;
```